

Documentación técnica del proyecto LilyGO ECG

2026-06-14

1. Mapa jerárquico

```
ECG_TDisplay_ESP32.ino
|--- setup
| |--- rawBuffer.begin / filteredBuffer.begin
| |--- xQueueCreate(adcQueue, dspQueue, bleQueue, displayQueue)
| |--- startBleTask -> TaskBLE
| |--- startDisplayTask -> TaskDisplay
| |--- startDspTask -> TaskDSP
| '--- startAdcTask -> TaskADC
'--- loop -> vTaskDelay

TaskADC
'--- configureAdcContinuous -> adc_continuous_new_handle/config/start
'--- promedio OSR -> dspQueue

TaskDSP
|--- Biquad notch50/100/150
|--- rawBuffer / filteredBuffer
|--- bleQueue
'--- displayQueue

TaskBLE
|--- NimBLE server/service/characteristics
|--- CommandCallbacks -> streamMode / sendBuffer
'--- notifications desde bleQueue

TaskDisplay
'--- LovyanGFX ST7789 desde displayQueue

Python ecg_ble_viewer.py
|--- BleakScanner/BleakClient
|--- Tkinter GUI
```

```
|--- ECG temporal
|--- FFT tiempo real
'--- grabación CSV temporizada
```

2. Dependencias cruzadas

Módulo	Depende de / entrega a
<code>config.h</code>	Define pines, tasas, tamaños de cola, UUIDs, estructuras y handles globales usados por todos los módulos de firmware.
<code>adc_task.cpp</code>	Recibe muestras de <code>esp_adc/adc_continuous</code> ; entrega <code>DecimatedSample</code> a <code>dspQueue</code> .
<code>dsp_task.cpp</code>	Recibe <code>DecimatedSample</code> ; entrega <code>ProcessedSample</code> a BLE, <code>display</code> y buffers circulares.
<code>ble_task.cpp</code>	Recibe comandos BLE y muestras procesadas; depende de <code>sample_buffer</code> para snapshots.
<code>display_task.cpp</code>	Consume la última muestra procesada por <code>displayQueue</code> ; depende de LovyanGFX.
<code>sample_buffer.cpp</code>	Recibe muestras de DSP; entrega snapshots cronológicos a BLE.
<code>ecg_ble_viewer.py</code>	Consume notificaciones BLE; entrega visualización temporal, FFT, energía por frecuencia y CSV.

3. Archivos y funciones

3.1. Firmware

ECG_TDisplay_ESP32.ino Punto de entrada Arduino. Existe para inicializar hardware, buffers, colas FreeRTOS y tareas. `setup()` no recibe parámetros y no retorna valor; crea los recursos globales. `loop()` sólo duerme porque el trabajo vive en tareas FreeRTOS.

config.h Cabecera central. No ejecuta lógica; entrega constantes, estructuras `AdcBlock`, `DecimatedSample`, `ProcessedSample`, enumeración `StreamMode` y declaraciones globales. Todos los módulos dependen de este archivo.

adc_task.h/cpp `startAdcTask()` crea `TaskADC`. `configureAdcContinuous()` configura DMA ADC1 canal 0, atenuación 12 dB y 12 bits. `taskAdc()` recibe tramas DMA, acumula `OVERSAMPLE_FACTOR` muestras, emite una media e incrementa índice.

dsp_task.h/cpp Biquad implementa un filtro IIR de segundo orden en forma directa II transpuesta. `process(x)` recibe una muestra flotante y entrega una muestra filtrada. `clampToInt16()` satura el resultado. `taskDsp()` centra la muestra ADC, aplica tres notch, actualiza buffers y colas. `startDspTask()` crea `TaskDSP`.

ble_task.h/cpp Usa NimBLE-Arduino. `ServerCallbacks` mantiene estado de conexión y reinicia advertising. `CommandCallbacks` interpreta comandos ASCII. `notifyText()` entrega respuestas cortas. `sendBuffer()` transmite snapshots. `taskBle()` crea servidor, servicio, características y empaqueta muestras para notificación. `startBleTask()` crea `TaskBLE`.

display_task.h/cpp Usa LovyanGFX. LGFX encapsula panel ST7789 y bus SPI. `taskDisplay()` inicializa backlight/TFT, muestra estado BLE y dibuja ECG filtrado. `startDisplayTask()` crea `TaskDisplay`.

sample_buffer.h/cpp `CircularSampleBuffer::begin()` limpia el buffer. `push(sample)` inserta una muestra con sección crítica. `snapshot(dest,maxSamples)` copia en orden cronológico hasta el máximo solicitado. `capacity()` entrega la capacidad.

3.2. Aplicación Python

python_app/ecg_ble_viewer.py GUI de investigación. `EcgBleApp` coordina Tkinter, Bleak, almacenamiento y gráficos. `scan/connect/disconnect/send` gestionan BLE. `_notification` decodifica `int16_t` little-endian. `_draw` refresca ECG temporal y FFT. `_compute_fft` aplica ventana Hann y FFT real con NumPy. `start_timed_recording` y `finish_timed_recording` implementan grabación automática de 1 o 5 minutos. `export_csv/write_csv` guardan muestras.

python_app/requirements.txt Declara `bleak` para BLE y `numpy` para FFT.

4. Librerías utilizadas

- Arduino core ESP32: arranque, GPIO, FreeRTOS y compatibilidad Arduino.
- ESP-IDF ADC continuous: adquisición ADC por DMA.
- FreeRTOS: tareas, colas, delays y secciones críticas.
- NimBLE-Arduino: servidor BLE GATT y notificaciones.
- LovyanGFX: control de pantalla ST7789 por SPI.
- Tkinter: interfaz de escritorio Python.
- Bleak: cliente BLE multiplataforma.
- NumPy: FFT y operaciones vectoriales.